

Accelerating SystemVerilog Assertion Proofs with LLMs and Machine Learning

Ilay Menahem

August 15, 2026

1 Introduction

Formal Property Verification (FPV) establishes whether a SystemVerilog design satisfies a SystemVerilog Assertion (SVA). Its cost, however, grows quickly with the size of the design’s state space. Verification engineers commonly use two complementary strategies to control this cost:

- **Hierarchical lemma generation:** construct a DAG of assertions rooted at the target, with helper assertions as descendants. Assuming proved child properties should make each parent assertion easier to verify, which decomposes a difficult proof into a sequence of tractable ones.
- **State-space reduction:** introduce cutpoints, abstractions, and blackboxing to shrink the space the verifier must explore.

We propose an LLM agent that interacts with the verifier in a closed loop. At each step, it adds helper assertions, cutpoints, abstractions, or blackboxes, then uses the verifier’s response to choose its next action.

We ask whether such agent can reduce the time or compute needed to prove valid SVA properties compared with direct proving and verifier-feedback prompting. We also test whether it can prove properties that these baselines cannot handle within the available budget.

The approach resembles mathematical proof in Lean, but SVA does not require a complete derivation from axioms or a formal derivation of every lemma from its dependencies. It also supplies more than a binary success or failure signal: the time required to discharge an assertion provides a graded measure of proof quality.

These differences suggest that LLM-based helper-assertion generation may be at least as feasible as LLM-based proof generation in Lean, where reinforcement learning has already produced strong results. AlphaProof reached silver-medal performance on IMO problems [7]. The DeepSeek-Prover series trained on proof-assistant feedback [27] and later applied reinforcement learning to subgoal decomposition [22], a task closely related to generating helper assertions.

2 Model

This work uses a standard LLM architecture, starting from a pretrained open-source model. The model must have agentic and reasoning capabilities, along with a context window larger than 128k tokens.

3 Data

3.1 Data sources

Source	Purpose	Verified labels	Used for
HierSVA [2]	Hierarchical contracts	Yes	SFT/RL
OpenTitan [12]	Assume-guarantee structure	Yes	SFT/RL
HWSec-UNC [1]	Security properties	Yes	SFT/RL
riscv-formal [25]	Compositional verification	Yes	SFT/RL
VERT [16]	SVA syntax breadth	Mixed	SFT
Verixa [26]	Verification context	Mixed	SFT
NVIDIA CVDP [18]	Specification and RTL	Mixed	SFT
VerilogEval [10]	English and RTL	No	SFT

Table 1: Fine-tuning and reinforcement-learning data sources.

3.2 Data augmentation

We augment each verified pair of a specification and contract in two ways. For identifier renaming, we replace signal, port, and module names with sampled alternatives that follow the design’s naming conventions. For syntactic paraphrasing, we apply rewrite rules to generate semantically equivalent SystemVerilog and SVA variants by reordering conjuncts or using De Morgan’s laws and equivalent temporal operators.

3.3 Designs and assertions for the environment

Each instance consists of a design and a target assertion. We include only assertions known to hold, which lets us attribute failures to the agent.

We use design-level Training, Validation, and Testing partitions. The partitions are the same for fine-tuning and RL. Every assertion, generated variant, and proof trace derived from a design stays in that design’s partition.

4 Harness

The full harness includes

- The prompt generation will include a skill file to help the LLM methodologically work like a verification engineer.

- Use lean-context to manage context. The agent must have permissions to read the design, assertion, and .tcl files. It should be able to write new assertion, auxiliary code, and .tcl files. The .tcl file contains the assume-and-prove commands and the abstractions.

5 Environment

5.1 The environment

The agent performs best-first search in an environment with the following mechanics:

- **Initialization.** An episode starts from a SystemVerilog design and a target assertion to prove.
- **Action.** At each step the agent emits one typed action:
 - `ADD_LEMMA(property, lemma_assertion);`
 - `ADD_CUTPOINT(signal);`
 - `ADD_BLACKBOX(module);`
 - `ADD_COUNTER_ABSTRACTION(module);`
 - `RETRACT(property);`

The environment rejects actions that violate the schema, contain invalid SVA, reference unknown design objects, or introduce a cycle.

- **Soundness.** The environment tracks the design, proof DAG, transformation history, and status of every obligation, and enforces:
 1. a property may use a descendant as an assumption only after that descendant is proved;
 2. benchmark-supplied assumptions are external premises; assumptions from generated interface contracts become obligations discharged at the enclosing scope;
 3. cutpoints, abstractions, and blackboxes produce explicit equivalence or interface obligations and are usable only once those are proved;
 4. failed or timed-out obligations cannot support an ancestor;
 5. the target is accepted only when its entire dependency closure is proved.
- **Observation.** After each step the agent sees the full proof state, serialized as text:
 - the design source and the target property;
 - the proof DAG: each obligation, its dependencies, and its status (proved, failed, or timed out);
 - the applied transformations and the obligations they introduced;
 - prover feedback for each failed obligation, either a counterexample trace projected onto the property’s cone of influence or the timeout bound reached;
 - the rejection reason, if the last action was invalid.

The design is pruned to the cone of influence of the open obligations, and counterexample traces are truncated to the cycles around the violation.

- **Termination.** The episode ends when the target is proved or a step limit is reached.

5.2 Prover flow

The environment requires a prover that parses SVA, discharges assertions unattended at scale, and supports cutpoints, abstractions, and blackboxing. We consider four candidates:

- **SymbiYosys + Verific (Tabby CAD Suite)** [29]: full IEEE 1800 SVA via the Verific frontend; cutpoints and blackboxing map directly to Yosys passes; BMC, k-induction, and IC3 engines. Proprietary but free for academic use.
- **SymbiYosys (open-source frontend)** [29]: fully open source, but parses only a subset of SVA, excluding the temporally deep properties in our corpus.
- **EBMC** [17]: word-level BMC with k-induction; broader SVA coverage than open-source Yosys and trivial to deploy on a cluster, but weaker engine variety and no built-in cutpoint support.
- **Commercial tools (e.g., JasperGold, VC Formal)**: full SVA support and the most capable engines, but license seats constrain the parallel proof workers required by the RL loop.

We use SymbiYosys with Verific because it is the only free option that combines full SVA parsing, unattended operation, and native support for cutpoints and blackboxes. EBMC serves as a fallback for designs outside the Verific flow.

5.3 Reward design

The task reward at step t is

$$r_t = R_{\text{solve}} \cdot 1_{\text{proved}} - \alpha c_t$$

where $c_t = \tilde{T}_t$ is the normalized prover time, and R_{solve} is the reward for proving the property. $R_{\text{solve}} \gg \alpha c_t$.

6 Fine-tuning

Before reinforcement learning, we fine-tune the base LLM on a domain corpus to teach it SystemVerilog, SVA, and the structure of assume-guarantee contracts (Section 7).

We first fine-tune on the full corpus, then run a second fine-tuning stage using only verified contracts.

7 Reinforcement learning

We train the fine-tuned LLM with either GRPO or PPO [24] with GAE [23].

7.1 Curriculum learning

Difficulty grading. Each pair receives a static difficulty proxy based on the cone-of-influence state-bit count, hierarchy depth, and the property’s temporal depth (the nesting of ##, |->, and unbounded operators). We calibrate the proxy against the baseline single-shot proof time and timeout rate, then divide the instances into tiers.

We use adaptive curriculum learning [3] to introduce more complex instances gradually.

8 Experiments

We evaluate whether the trained agent proves more properties at lower cost than direct proving and prompting-based alternatives. The test set is the held-out portion of the verified corpus (Section 3.3).

8.1 Methods compared

- **Prover-only. (baseline)** SymbiYosys/Verific run directly on the target assertion with the full budget and no transformations.
- **Frozen LLM.** The pretrained base model in the environment.
- **Frozen LLM + Full harness.** The pretrained base model in the environment, with a full harness (Section 4).
- **Full agent.** The fully trained policy in the environment with the full harness (Section 4).

8.2 Metrics

Effectiveness.

- *Solve rate:* fraction of instances whose target and full obligation closure are proved within budget, overall and per tier.
- *Newly solved instances:* targets proved that prover-only cannot discharge within the same budget.

Efficiency (on instances solved by both methods being compared).

- *End-to-end wall-clock time* to a completed sound proof; median and speedup relative to prover-only.
- *Verifier time and calls:* cumulative prover CPU-time and obligations dispatched.
- *Compute per solved target:* CPU- and GPU-hours per solve, including the LLM’s computational cost.

Proof and policy quality.

- *Proof-DAG size and depth:* helper properties and obligations in the accepted proof; smaller DAGs are more targeted and cheaper to re-check.
- *Action validity rate:* fraction of actions accepted by the environment schema and design checks.
- *Obligation discharge rate:* fraction of introduced obligations eventually proved, separating productive lemmas from speculative ones.

8.3 Failure analysis

Unsolved instances are categorized by terminal state: prover timeout on a valid decomposition, step-budget exhaustion without a viable DAG, persistent invalid actions, or unproductive loops with repeated lemma and retract cycles.

9 Implementation plan

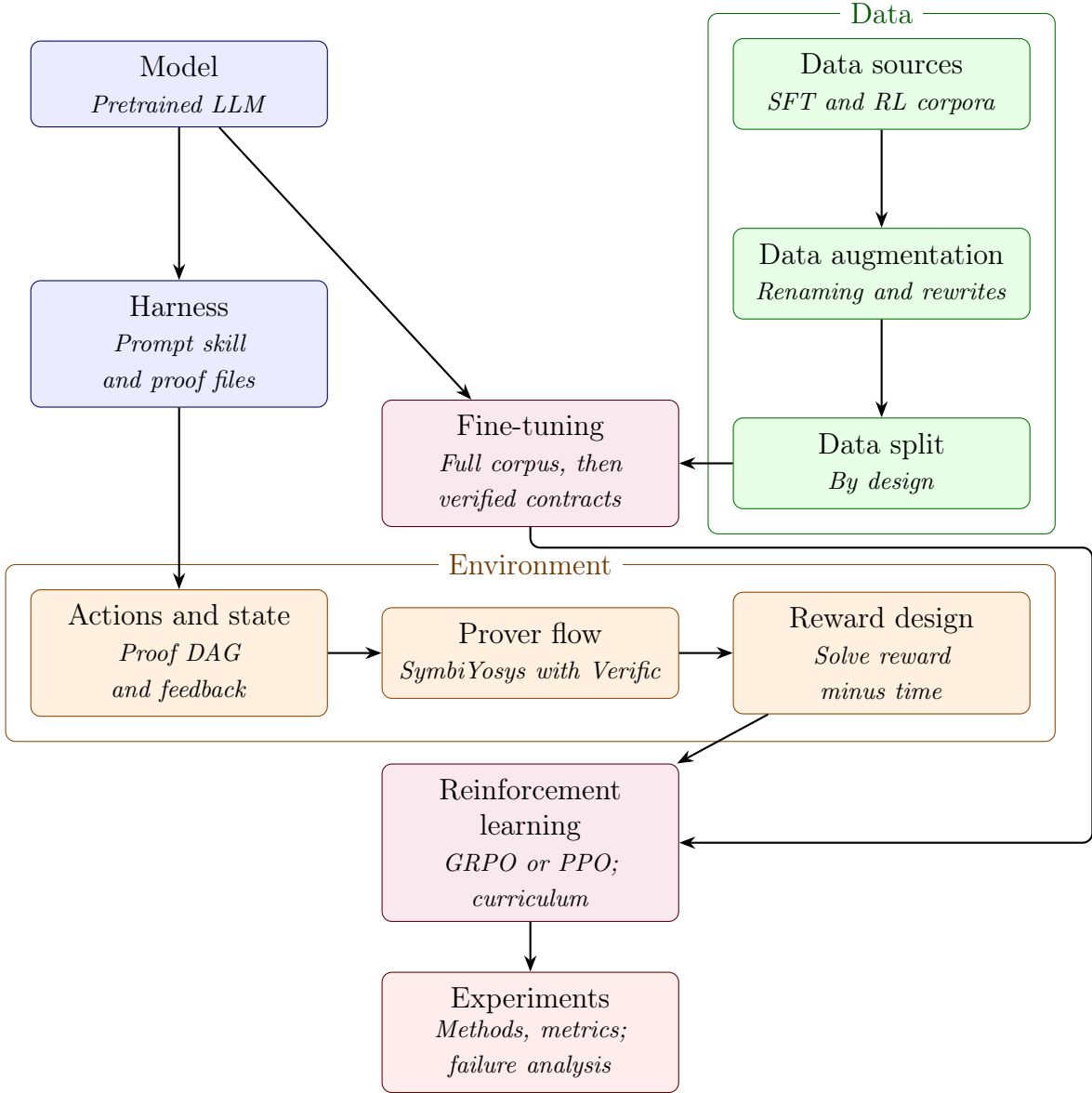


Figure 1: Implementation dependencies from the model and data to the experiments. The graph expands data preparation and the environment into separate stages.

A Related work

A.1 LLMs for formal proof

Neural theorem provers such as GPT-f [20], LeanDojo [28], DeepSeek-Prover [27, 22], and AlphaProof [7] combine an LLM policy with a sound verifier, then reuse successful trajec-

Quantity	Value	Rationale
Episode step limit	5 actions	Matches the re-prompting limit of [19].
Prover timeout per obligation	30 s	Matches the per-query timeout of [19].
Test wall-clock budget	2 h per instance	Identical for every method compared.

tories for training. We apply this closed-loop approach to FPV, where verifier runtime and discharged proof obligations provide graded feedback.

A.2 LLMs for hardware verification

Most LLM work in hardware targets RTL generation [10, 11], assertion generation [5, 13], or testbench generation [21], rather than proving a given property. The closest prior work is the lemma-mining framework of Peled et al. [19], in which an LLM proposes invariants and a model checker selects an inductive strengthening. Our work instead trains the policy using supervised fine-tuning and reinforcement learning and targets hierarchical assume-guarantee decomposition together with verified cutpoints, abstractions, and blackboxing.

A.3 Classical FPV techniques

Hardware model checking already uses localization abstraction [9], CEGAR [4], predicate and proof-based abstraction [8, 15], and word-level abstraction [6, 14]. In industrial FPV, engineers also create helper lemmas and module-level assume-guarantee contracts. Our contribution is not a new proof rule, but an agent that learns to apply these sound, established transformations in a closed loop with the verifier.

A.4 Data and evaluation gap

Existing datasets are primarily generation-oriented: VERT [16], Verixa [26], and NVIDIA CVDP [18] provide SVA or verification examples but do not supply verified hierarchical proof trajectories. Peled et al. [19] provide 109 flat SystemVerilog proof tasks. We therefore construct design-level splits with verified assume-guarantee examples and replayable proof trajectories (Section 6).

References

- [1] Hammad Ahmad, Yu Huang, and Westley Weimer. Security properties for open-source hardware designs. *arXiv preprint arXiv:2412.08769*, 2024.
- [2] Anonymous. HierSVA: A data synthesis pipeline, dataset, and benchmark for hierarchical SystemVerilog assertions. Hugging Face dataset <https://huggingface.co/datasets/AnonymousHierSVA/HierSVA>, 2026.

- [3] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48. ACM, 2009.
- [4] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV)*, 2000.
- [5] Wenji Fang, Mengming Wang, Yao Lu, Shang Liu, Hongce Zhang, and Zhiyao Xie. AssertLLM: Generating hardware verification assertions from design specifications via multi-LLMs. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2025.
- [6] Aman Goel and Karem Sakallah. AVR: Abstractly verifying reachability. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2020.
- [7] Google DeepMind AlphaProof and AlphaGeometry Teams. AI achieves silver-medal standard solving international mathematical olympiad problems. Google DeepMind Blog, 2024.
- [8] Susanne Graf and Hassen Saïdi. Construction of abstract state graphs with PVS. In *Computer Aided Verification (CAV)*, 1997.
- [9] Robert P. Kurshan. *Computer-Aided Verification of Coordinating Processes: The Automata-Theoretic Approach*. Princeton University Press, 1994.
- [10] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [11] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. RTL-Coder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution. *arXiv preprint arXiv:2312.08617*, 2024.
- [12] lowRISC. OpenTitan: Open source silicon root of trust. <https://github.com/lowRISC/opentitan>, 2024.
- [13] Bhabesh Mali, Karthik Maddala, Sweeya Reddy, Vatsal Gupta, Chandan Karri, and Subhasis Roy. ChIRAAG: ChatGPT informed rapid and automated assertion generation. *arXiv preprint arXiv:2402.00093*, 2024.
- [14] Makai Mann, Ahmed Irfan, Florian Lonsing, Yahan Yang, Hongce Zhang, Kristopher Brown, Aarti Gupta, and Clark Barrett. Pono: A flexible and extensible SMT-based model checker. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2021.
- [15] Kenneth L. McMillan and Nina Amla. Automatic abstraction without counterexamples. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2003.

- [16] Anand Menon, Vatsal Gupta, Chandan Karri, Tajana Rosing, and Kanad Basu. Enhancing large language models for hardware verification: A novel SystemVerilog assertion dataset. *arXiv preprint arXiv:2503.08923*, 2025.
- [17] Rajdeep Mukherjee, Peter Schrammel, Daniel Kroening, and Tom Melham. Unbounded safety verification for hardware using software analyzers. In *Design, Automation & Test in Europe (DATE)*, 2016.
- [18] NVIDIA. CVDP: Comprehensive Verilog design problems benchmark dataset. Hugging Face dataset <https://huggingface.co/datasets/nvidia/cvdp-benchmark-dataset>, 2025.
- [19] Romy Peled, Daniel Kroening, Michael Tautschnig, and Yakir Vizel. Large lemma miners: Can LLMs do induction proofs for hardware? *arXiv preprint arXiv:2511.02521*, 2026.
- [20] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- [21] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*, 2024.
- [22] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- [23] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- [24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [25] SymbioticEDA. riscv-formal: RISC-V formal verification framework. <https://github.com/SymbioticEDA/riscv-formal>, 2024.
- [26] Vyoma Labs. Verixa dataset: Hardware verification, SVA, UVM, and formal verification corpus. Hugging Face dataset <https://huggingface.co/datasets/vyomaalabs/verixa-dataset>, 2025.
- [27] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, et al. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. *arXiv preprint arXiv:2408.08152*, 2024.

- [28] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- [29] YosysHQ. SymbiYosys (sby): Front-end for Yosys-based formal verification flows. <https://github.com/YosysHQ/sby>, 2024.